Persistent

Agenda

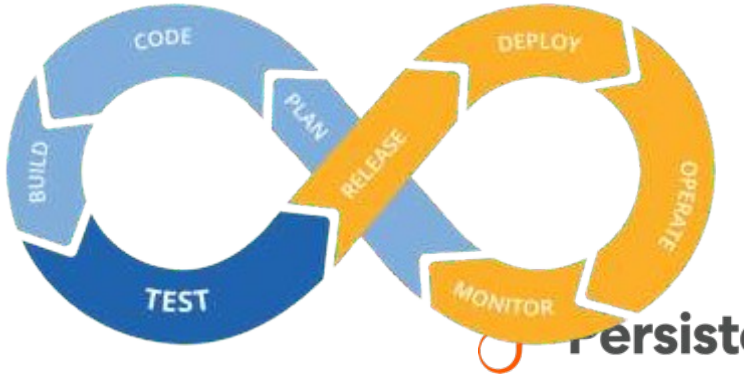
- What is CI-CD
- Tools for CI-CD
- Why GitLab CI-CD
- What is GitLab CI-CD
- Gitlab CI-CD features
- Use Case & Best Practices
- Demo

What is CI -CD

- **Continuous Integration (CI) :** Continuous Integration is the consistent and automated practice of compiling, testing, and merging code changes into a shared code repository.

- **Continuous Delivery (CD) :** It is the automated procedure of deploying software updates to either a staging or production environment after they have been effectively built and tested.

- **Continuous Deployment (CD) :** Continuous Deployment resembles Continuous Delivery but uniquely involves the automatic release of changes to the production environment as soon as they are deemed ready, without an intermediary staging phase.



Tools for CI-CD

	Jenkins	GitLab	Circle CI	Bamboo	Teamcity
Open Source	YES	NO	NO	NO	NO
Hosting	ON-PREMISE & CLOUD	ON-PREMISE & CLOUD	ON-PREMISE	ON-PREMISE & BIT-BUCKET as CLOUD	ON-PREMISE & CLOUD
Easy to Setup	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM
Free Version	YES	YES	YES	YES	YES

Comparison Table: Jenkins vs GitLab CI vs CircleCI vs Bamboo vs TeamCity

Feature / Tool	**Jenkins**	**GitLab CI**	**CircleCI**	**Bamboo** (Atlassian)	**TeamCity** (JetBrains)
Type	Open Source	Open Source + SaaS	SaaS + Self-hosted	Commercial	Commercial
Ease of Setup	Complex (manual plugin setup)	Easy (integrated into GitLab)	Very Easy (SaaS)	Medium (on-premises)	Medium (on-premises)
UI/UX	Basic (improved with Blue Ocean)	Modern, integrated UI	Modern, clean UI	Good UI	Excellent UI with JetBrains quality
Pipeline as Code	Yes (Jenkinsfile)	Yes (.gitlab-ci.yml)	Yes (config.yml)	Yes (YAML + GUI)	Yes (Kotlin DSL, also GUI based)
Scalability	Very High (plugin-based, master-agent)	High (Auto-scaling runners)	High (Docker-based, parallel jobs)	Medium (agent-based)	Medium (agent-based)

Comparison Table: Jenkins vs GitLab CI vs CircleCI vs Bamboo vs TeamCity

Feature / Tool	Jenkins	GitLab CI	CircleCI	Bamboo (Atlassian)	TeamCity (JetBrains)
Docker Support	Strong (via plugin)	Native	Excellent (Docker-first)	Good (Docker agent)	Good (Docker and Kubernetes support)
Kubernetes Support	Yes (via plugin)	Yes (via GitLab K8s executor)	Yes (orbs or custom runners)	Limited (not native)	Yes (K8s plugin or custom runners)
Parallel Execution	Yes (manual config with agents)	Yes (matrix jobs, dynamic pipelines)	Yes (easy parallelism setup)	Yes	Yes
Secrets Management	Plugin-based (Vault, credentials)	Built-in (masked CI/CD variables)	Built-in (env vars, contexts)	Built-in	Built-in
Plugin Ecosystem	Very large (~1800+)	Smaller, focused	Small, focused	Limited	Moderate
Notifications	Via Plugins (Slack, Email, etc.)	Built-in	Built-in (Slack, Email)	Built-in	Built-in (customizable)
Test Reporting	Via Plugins	Built-in (JUnit, Code coverage)	Built-in	Built-in	Built-in

Comparison Table: Jenkins vs GitLab CI vs CircleCI vs Bamboo vs TeamCity

Feature / Tool	Jenkins	GitLab CI	CircleCI	Bamboo (Atlassian)	TeamCity (JetBrains)
Test Reporting	Via Plugins	Built-in (JUnit, Code coverage)	Built-in	Built-in	Built-in
Manual Approval / Gates	Plugin-based	Built-in (Environments / approvals)	Built-in (with workflows)	Built-in	Built-in
Security	Requires hardening	Good (RBAC, project/group level)	Good (context-based)	Good (integrates with LDAP/SSO)	Good (LDAP, RBAC)
Maintenance Overhead	High (self-managed)	Low (SaaS) to Medium (self-hosted)	Very Low (SaaS)	Medium to High	Medium
Cost	Free (self-hosted)	Free tier + paid runners	Free tier + paid usage	Paid	Paid (free tier for OSS)
Best for	Custom, complex pipelines	GitLab users; integrated DevOps	Quick CI/CD setup, Docker-heavy apps	Jira/Bitbucket users	JetBrains ecosystem or .NET-heavy teams

What is Gitlab CI-CD

GitLab CI/CD is a built-in Continuous Integration and Continuous Deployment (CI/CD) tool provided by GitLab, a popular web-based Git repository manager.

- GitLab CI/CD automates building, testing, and deploying code changes.
- CI/CD pipelines are defined using a version-controlled **.gitlab-ci.yml** file.
- GitLab CI/CD is tightly integrated with GitLab's collaboration and source
  code management features.
- It supports parallel job execution, speeding up testing and deployment processes.

Persistent

Key Components of Gitlab CI-CD

GitLab CI/CD consists of several key components that work together to automate the software development lifecycle. The main components are :

- **Runner**
- **Pipeline**
- **Gitlab CI file**
- **Artifacts**
- **Triggers**
- **CI-CD Variables**

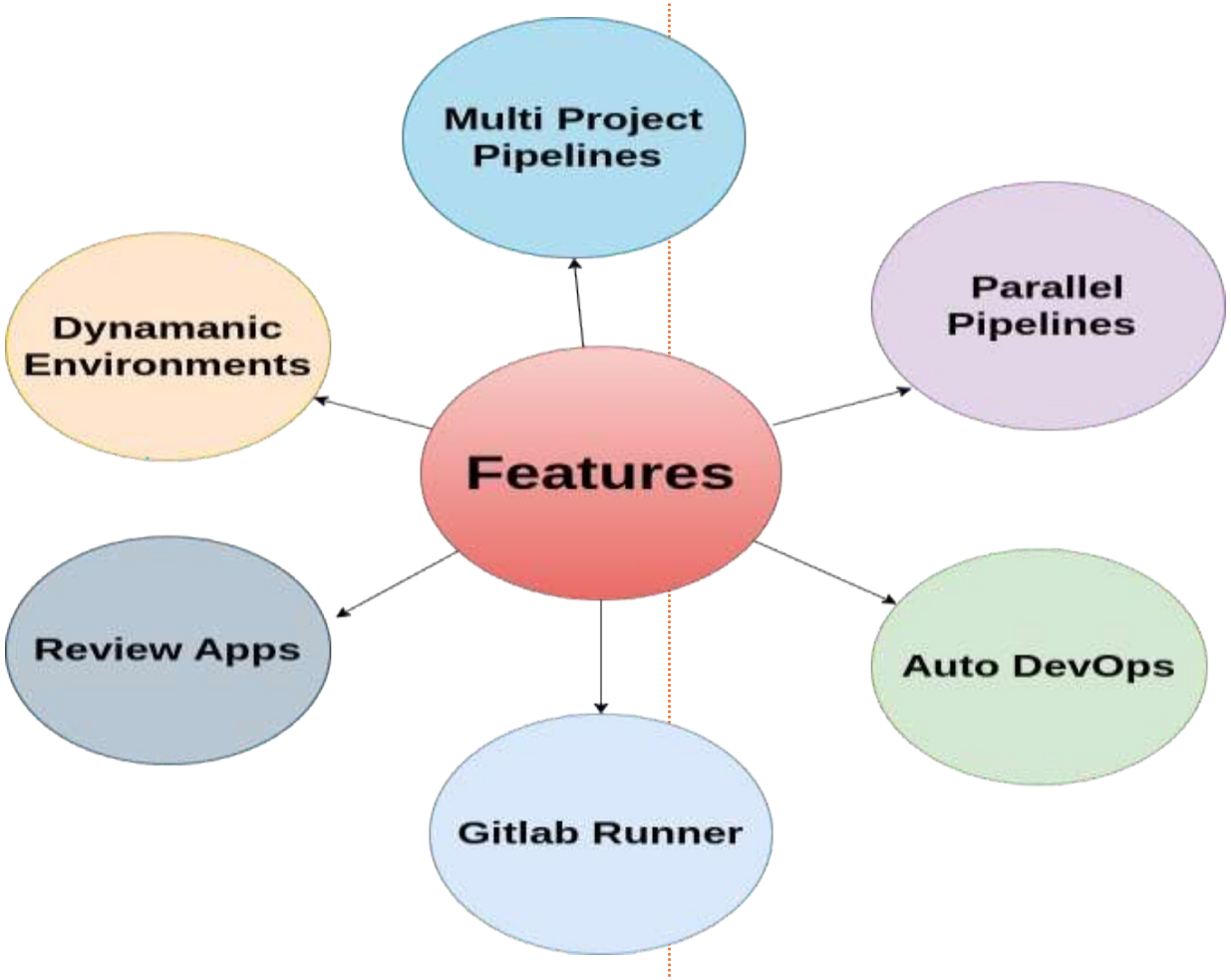
Persistent

Why Gitlab CI-CD

GitLab CI/CD is a popular choice for many development teams and organizations for several reasons:

- **Automated and Consistent Build and Deployment Process**
- **Faster Time-to-Market**
- **Continuous Integration and Testing**
- **Improved Collaboration**
- **Enhanced Security and Stability**

Gitlab CI-CD features



Persistent

Use Case and Best practices

Use Case -

- Continuous integration and deployment for web applications
- Infrastructure automation
- Data pipelines
- Automated Testing
- Containerized Applications

Best Practise -

- Security and secrets management
- Parallelization and caching
- Infrastructure as code
- Pipeline testing

Persistent

DEMO

Thank you

For more information and any queries contact to Deeksha.Tripathi@*nashtechglobal.com*

Nash Tech.

Persistent